Sir M Visvesvaraya Institute of Technology

Bengaluru-562 157

Department of Electrical and Electronics Engineering

Arduino Project Report (BEEL456D)

Sem: IV Section:A AY:2024-25

Objective: To develop skills required to build IoT based projects.

Title of Project: IoT-Based Voice-Activated Home Automation System

Submitted by

Name:	USN:
ASTITWA SINGH	1MV23EE011
MANOHAR KUMAR	1MV23EE034
MAYANK KUMAR	1MV23EE035
NITESH KUMAR SAH	1MV23EE046

Abstract:

This project presents a Voice-Controlled Home Automation System using an ESP8266 Wi-Fi module(NodeMCU), DHT11 temperature sensor, LDR light sensor. The system enables voice-based control of household appliances like fans and lights by a smartphone app with voice assistant integration. The ESP8266 connects to Wi-Fi and controls relays to turn on/off devices based on voice commands . The DHT11 monitors temperature, while the LDR detects light levels to automate appliance control. This setup combines convenience, automation, and real-time sensor data for an efficient home automation solution

Working of the Circuit:

1. Voice-Based Control by NodeMCU:
   - Google Assistant → Sinric Pro → NodeMCU
   - When we say:
     - "Turn ON the fan" → NodeMCU sends "FAN_ON" via Serial to Arduino
     - "Turn OFF the light" → NodeMCU sends "LIGHT_OFF" to Arduino
   - NodeMCU sets control mode to manual override, disabling sensor control
   - Relay CH1 and CH3 (for light & fan) switch ON/OFF as per command

---

2. Sensor-Based Automatic Control via Arduino:

   - LDR reads light level; DHT11 reads temperature
   - When no voice command has been given, Arduino controls:
     - Fan: ON if temperature > threshold (e.g., 30°C)
     - Light: ON if ambient light < threshold

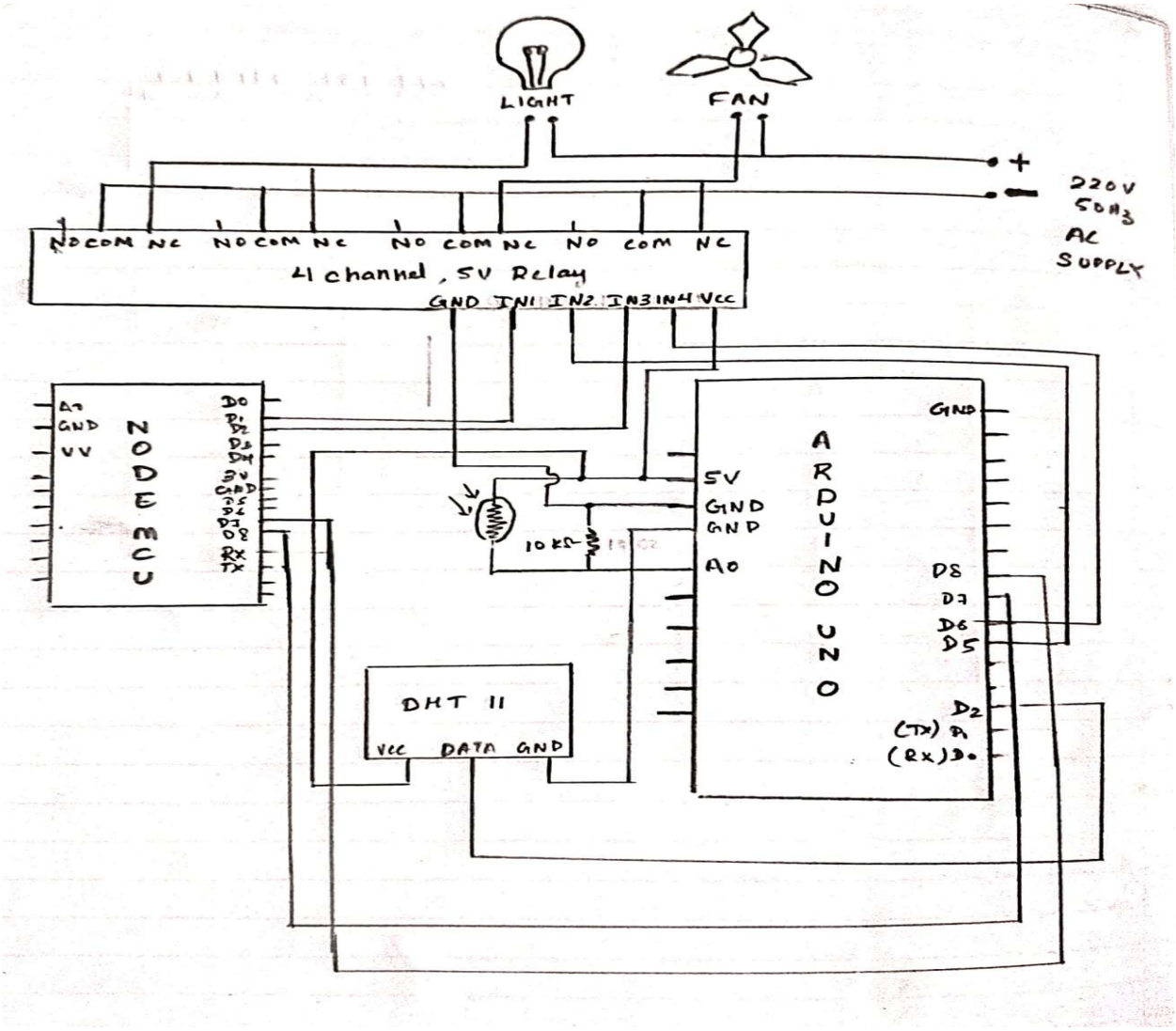
- Relay CH2 and CH4 are used by Arduino for auto control

3. Priority Handling:
   - When a voice command is received:
     - Arduino sets manualFanControl = true or manualLightControl = true
     - This overrides sensor decisions — fan/light will stay ON/OFF as commanded
   - Voice command "FAN_AUTO" or "LIGHT_AUTO" resets control to automatic mode
   - So, even if temp is high, if you said “Turn OFF fan”, it stays OFF

4.Relay Channel Assignment (4-Channel Relay):

Channel	Controlled By	Device	Mode
CH1	NodeMCU	Light	Voice control
CH2	Arduino	Light	Sensor-based
CH3	NodeMCU	Fan	Voice control
CH4	Arduino	Fan	Sensor-based

Connection Diagram:



Approximate cost of project:

Component	Cost in Rs
Arduino Uno	**₹550**
NodeMCU ESP8266	**₹200**
4-Channel Relay Module (5V)	**₹150**
DHT11 Temperature Sensor	**₹80**
LDR (Light Dependent Resistor)	**₹10**
10k Ohm Resistor	**₹2**
Breadboard + Jumper Wires	**₹150**
AC Bulb and Holder	**₹50**
AC Fan (or small motor)	**₹200**
Total Estimated Cost	**1400**

Program:

NodeMCU (ESP8266) Code – Voice Control (Sinric Pro):

```
#ifdef ENABLE_DEBUG
  #define DEBUG_ESP_PORT Serial
  #define NODEBUG_WEBSOCKETS
  #define NDEBUG
#endif
#include <Arduino.h>
#if defined(ESP8266)
  #include <ESP8266WiFi.h>
#elif defined(ESP32) || defined(ARDUINO_ARCH_RP2040)
  #include <WiFi.h>
#endif
#include "SinricPro.h"
#include "SinricProSwitch.h"
#define WIFI_SSID     "POCO C51"
#define WIFI_PASS     "12345678"
#define APP_KEY       "e3c4a688-5621-42ab-8605-f660dad95ffc"
#define APP_SECRET    "2e1e2690-19d1-4d20-8592-8e1f634f60a4-6904a14e-e4a9-400b-8bc8-
1c103a546f2c"
#define SWITCH_ID_1   "68135335dc4a25d5c3c0a786"
#define SWITCH_ID_2   "681cd741bddfc53e33f1f6d7"
#define RELAYPIN_1    5
#define RELAYPIN_2    4
#define BAUD_RATE     115200
bool onPowerState1(const String &deviceId, bool &state) {
   Serial.printf("Light turned %s\n", state ? "ON" : "OFF");
  digitalWrite(RELAYPIN_1, state ? HIGH : LOW);
   return true;
```

```
}

bool onPowerState2(const String &deviceId, bool &state) {
Serial.printf("Fan turned %s\n", state ? "ON" : "OFF");
 digitalWrite(RELAYPIN_2, state ? HIGH : LOW);
return true;
}
void setupWiFi() {
  Serial.println("\n[WiFi]: Connecting...");
  #if defined(ESP8266)
    WiFi.setSleepMode(WIFI_NONE_SLEEP);
    WiFi.setAutoReconnect(true);
  #elif defined(ESP32)
    WiFi.setSleep(false);
    WiFi.setAutoReconnect(true);
  #endif
 WiFi.begin(WIFI_SSID, WIFI_PASS);
 while (WiFi.status() != WL_CONNECTED) {
     Serial.print(".");
     delay(250);
}

   Serial.printf("\n[WiFi]: Connected. IP Address: %s\n", WiFi.localIP().toString().c_str());

}
void setupSinricPro() {
 pinMode(RELAYPIN_1, OUTPUT);
 pinMode(RELAYPIN_2, OUTPUT);
  digitalWrite(RELAYPIN_1, LOW);
  digitalWrite(RELAYPIN_2, LOW);
 SinricProSwitch& mySwitch1 = SinricPro[SWITCH_ID_1];
  mySwitch1.onPowerState(onPowerState1);
SinricProSwitch& mySwitch2 = SinricPro[SWITCH_ID_2];
mySwitch2.onPowerState(onPowerState2);
SinricPro.onConnected([]() {
   Serial.println("Connected to SinricPro");
 });
 SinricPro.onDisconnected([]() {
Serial.println("Disconnected from SinricPro");
  });
  SinricPro.begin(APP_KEY, APP_SECRET);
}
void setup() {
  Serial.begin(BAUD_RATE);
  Serial.println("\n\n--- Starting NodeMCU ---");
   setupWiFi();
setupSinricPro();
}
```

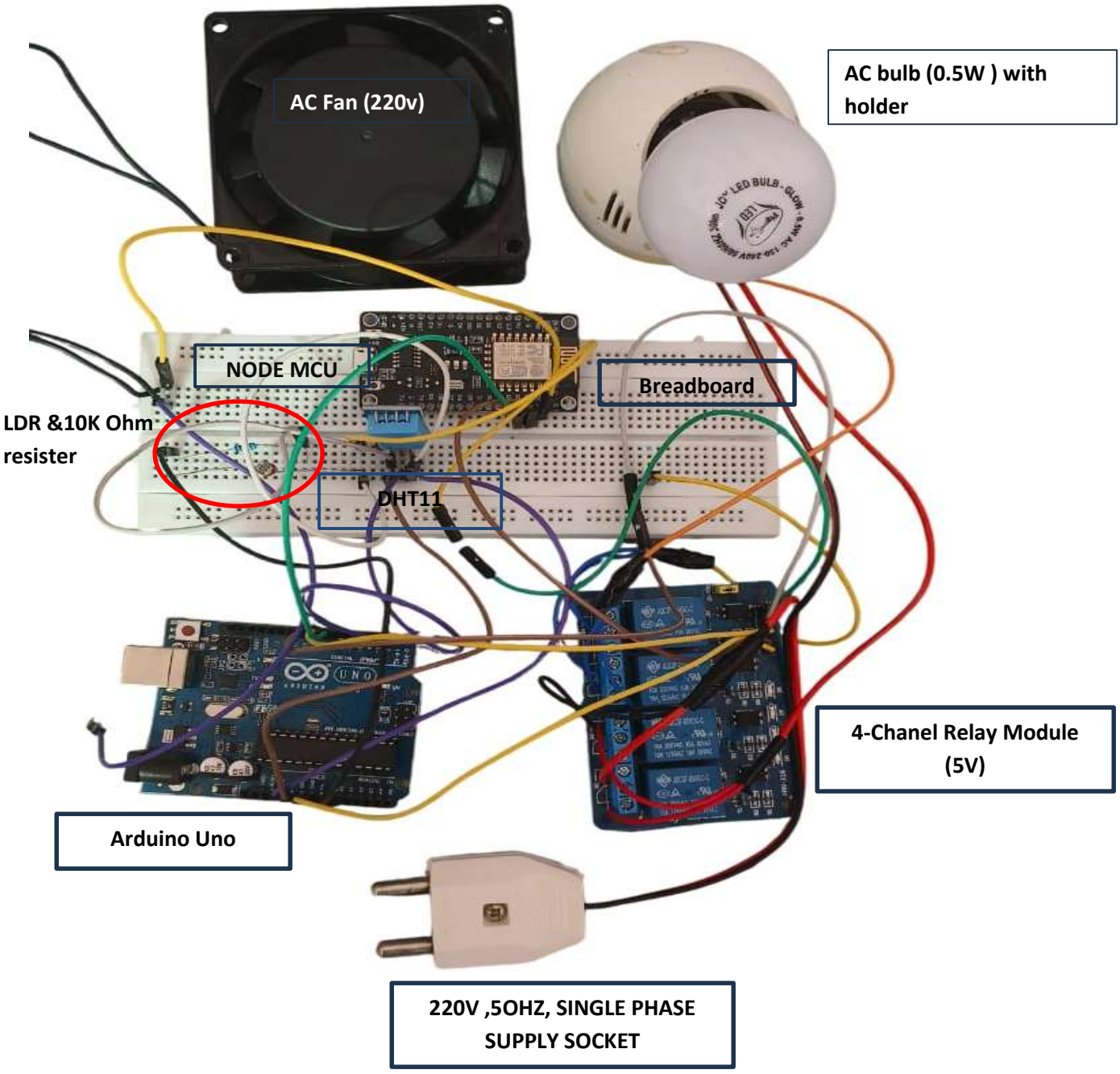
```
void loop() {
 SinricPro.handle();
}
```

Arduino Uno Code (Sensor-Based Control with NodeMCU Priority):

```
#include <DHT.h>
#define LDR_PIN       A0     // LDR sensor connected to analog pin A0
#define DHT_PIN       2      // DHT11 sensor connected to digital pin 2
#define DHT_TYPE      DHT11
#define LIGHT_RELAY   8      // Light relay connected to pin 8
#define FAN_RELAY     9      // Fan relay connected to pin 9
#define NODEMCU_LIGHT_STATUS 7   // Reads NodeMCU D7 (light priority signal)
#define NODEMCU_FAN_STATUS   6   // Reads NodeMCU D8 (fan priority signal)
DHT dht(DHT_PIN, DHT_TYPE);
void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(LIGHT_RELAY, OUTPUT);
  pinMode(FAN_RELAY, OUTPUT);
  pinMode(NODEMCU_LIGHT_STATUS, INPUT);
  pinMode(NODEMCU_FAN_STATUS, INPUT);
  digitalWrite(LIGHT_RELAY, LOW);  // Initially OFF
  digitalWrite(FAN_RELAY, LOW);    // Initially OFF
}
void loop() {
  bool lightPriority = digitalRead(NODEMCU_LIGHT_STATUS);
  bool fanPriority   = digitalRead(NODEMCU_FAN_STATUS);
  int ldrValue = analogRead(LDR_PIN);
  Serial.print("LDR: "); Serial.println(ldrValue);
 if (!lightPriority) {
    if (ldrValue < 500) {
      digitalWrite(LIGHT_RELAY, HIGH);  // Turn on light (it's dark)
    } else {
      digitalWrite(LIGHT_RELAY, LOW);   // Turn off light
    }
  }
  float temperature = dht.readTemperature();
  Serial.print("Temp: "); Serial.println(temperature);
    if (!fanPriority) {
    if (temperature > 30.0) {
      digitalWrite(FAN_RELAY, HIGH);  // Turn on fan (it's hot)
    } else {
```

```
    digitalWrite(FAN_RELAY, LOW);   // Turn off fan
   }
  }
   delay(2000); // Wait before next reading
}
```

Hardware model (Photo):



Learning Outcome:

- Gained hands-on experience in developing an IoT-based smart home system integrating both Arduino Uno and NodeMCU (ESP8266) microcontrollers.
- Learned how to implement sensor-based automation (using LDR for light control and DHT for temperature-based fan control) using Arduino.
- Understood how to use SinricPro with NodeMCU to enable voice-controlled automation via platforms like Amazon Alexa or Google Assistant.
- Explored inter-device communication between NodeMCU and Arduino using digital I/O pins to establish priority-based control.
- Strengthened skills in embedded systems programming, sensor interfacing, and real-time data processing.
- Practiced modular system design by dividing tasks between devices: Arduino handling autonomous sensor logic, NodeMCU handling cloud-based voice control.

Conclusion:

In this project we successfully implemented an IoT-based voice and sensor-controlled home automation system. The Arduino handles automatic control of lights and fans based on ambient light and temperature, ensuring efficient energy usage. Simultaneously, the NodeMCU enables remote voice control via the SinricPro cloud platform. Priority was given to NodeMCU, allowing users to override sensor control using voice commands, enhancing the system's flexibility and user control. This dual-control approach combines convenience with automation, showcasing the potential of integrating microcontrollers for smarter living environments.